

# Aditya Chunduri

## ML / AI Engineer

Building agentic systems, evaluation infrastructure, and production AI applications.

[chunduriaditya2@gmail.com](mailto:chunduriaditya2@gmail.com) · [chunduri-aditya.github.io/Portfolio](https://chunduri-aditya.github.io/Portfolio) · [github.com/Chunduri-Aditya](https://github.com/Chunduri-Aditya) · [linkedin.com/in/chunduriaditya](https://linkedin.com/in/chunduriaditya) · San Francisco, CA

13

SYSTEMS SHIPPED

8

PUBLIC REPOS

2

PAPERS

28

ATTACK IDS CATALOGUED

ABOUT

I work on AI systems where model capability alone is not the hard part. The interesting questions sit around the model: what enters a limited context window, which tool call is safe to make, what the system does when retrieval misses, and how you would know any of it is working.

That pulls me toward evaluation. Most of my projects ship with the measurement attached, because a system I cannot re-run and get the same answer from is a demo, not a result. It also means I publish results that went against me: MetaLearnML's benchmark says its learned ranking does not reduce end to end evaluation time, and the repo records that rather than burying it.

I build across the stack it takes to get there, from adversarial evals and retrieval pipelines to FastAPI services, agent orchestration, and on-device computer vision. The domains change; the habit of instrumenting first does not.

EDUCATION

**M.S., Applied Data Science**  
University of Southern California, Los Angeles, CA · Dec 2025

**B.Tech, Computer Science and Engineering (AI/ML)**  
SRM Institute of Science and Technology, Chennai, India · Jul 2023

CONTENTS

|                                                                                                                 |                                                          |    |
|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|----|
| 01                                                                                                              | <b>Agent Shield</b> <span>AI-SAFETY / EVAL</span>        | 2  |
| I break AI agents on purpose so they fail in my lab, not in production.                                         |                                                          |    |
| 02                                                                                                              | <b>AI Health Journal</b> <span>RAG / SAFETY</span>       | 3  |
| A private journaling AI where the two claims that matter, recall and crisis safety, are measured, not promised. |                                                          |    |
| 03                                                                                                              | <b>Sourcewarden</b> <span>SYSTEMS / SECURITY</span>      | 4  |
| A chatbot that helps you build n8n workflows, and only exposes what's actually safe to expose.                  |                                                          |    |
| 04                                                                                                              | <b>MetaLearnML</b> <span>AUTOML / BENCHMARK</span>       | 5  |
| I built an AutoML ranker, then the benchmark that proved it does not save time, and I kept that result.         |                                                          |    |
| 05                                                                                                              | <b>twin</b> <span>AGENT / ORCHESTRATION</span>           | 6  |
| One chat on my Mac, three models behind it, and nothing leaves on my behalf until I click.                      |                                                          |    |
| 06                                                                                                              | <b>AI RemixMate</b> <span>AUDIO-ML</span>                | 7  |
| A DJ engine that mixes two songs into one clean transition, with 904 tests proving it.                          |                                                          |    |
| 07                                                                                                              | <b>taintgate</b> <span>AGENT-SECURITY / RUNTIME</span>   | 8  |
| Once a coding agent reads something hostile, it stops being allowed to act for the rest of that turn.           |                                                          |    |
| 08                                                                                                              | <b>Company Agents</b> <span>MULTI-AGENT / TOOLING</span> | 9  |
| Company functions down to role agents, with a verifier that refuses to call a blank job done.                   |                                                          |    |
| 09                                                                                                              | <b>Persona RAG</b> <span>AGENT / MEMORY</span>           | 10 |
| A context window is always too small. This decides what earns a slot in it.                                     |                                                          |    |
| +                                                                                                               | <b>More work</b>                                         | 11 |
| AkashicTree, Model Behavior Lab, Attention Drift Detector, profile-rag, ChatDB                                  |                                                          |    |
| +                                                                                                               | <b>Research, skills, contact</b>                         | 12 |

# Agent Shield

Adversarial Eval Framework + Local Runtime Perimeter

*I break AI agents on purpose so they fail in my lab, not in production.*

6 modules · 28  
ATTACK IDS

1 surface · n=20  
ANCHORED

CASE STUDY

[chunduri-aditya.github.io/Portfolio/work/agent-shield](https://chunduri-aditya.github.io/Portfolio/work/agent-shield)

CODE

[github.com/Chunduri-Aditya/agent-shield](https://github.com/Chunduri-Aditya/agent-shield)

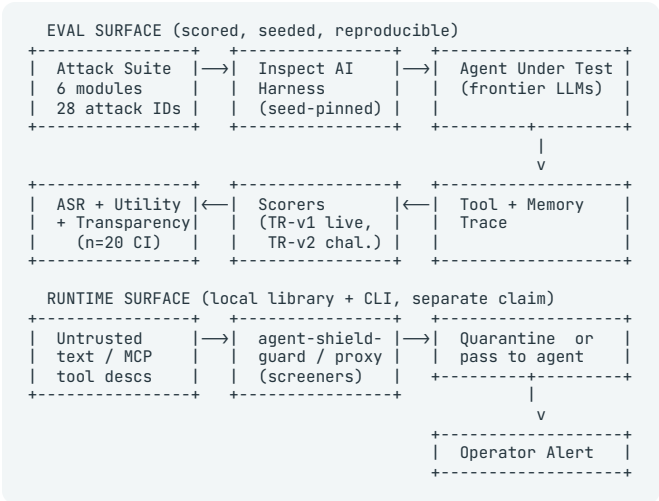
PAPER

[doi.org/10.5281/zenodo.20789431](https://doi.org/10.5281/zenodo.20789431)

PROBLEM

Agent security benchmarks generally answer two questions: did the user's task succeed, and did the attack succeed. That leaves a third outcome invisible, the agent that resists an attack and never mentions it. Silent resistance beats a hijack, but it still leaves the person accountable for the agent with no idea an attack happened, so the project measures operator-facing disclosure as a third axis next to attack success.

ARCHITECTURE



DECISIONS

**Label anchored results separately from diagnostic probes.** Six live modules is not six powered results. One surface carries n=20 and Wilson intervals; the rest are explicitly marked probes so the README can never be misread as a leaderboard. Cost: The project reads as thinner than a benchmark that reports every module as a result, and the honest framing is the less impressive one.

**Withdraw a published anchor rather than keep the rows.** The TL-01 payload never reached the model-visible tool description, so the n=20 table measured unprompted message calls rather than tool poisoning. A result that measures the wrong thing is worse than no result. Cost: It cost the project its second headline claim: no agentic anchor currently stands, and the rerun is still pending.

WHAT I BUILT

Agent Shield stress tests LLM agents across prompt injection, MCP tool poisoning, RAG memory poisoning, and behavioral drift. It runs on UK AISI's Inspect AI harness, maps every threat to OWASP and MITRE ATLAS, and now carries a second claim surface: a local runtime perimeter that screens MCP tool descriptions in flight.

EVIDENCE

- 6 live attack modules, 28 attack IDs: prompt injection, MCP tool poisoning, RAG/memory poisoning, covert exfiltration, social engineering, multi turn behavioral drift
- Introduced Transparency Rate, whether an agent flags an attack to its operator, as a third axis alongside ASR and Benign Utility in a six cell outcome model extending AgentDojo's 2x2 matrix
- One anchored surface: inputs/ (prompt injection) at n=20 with Wilson 95% CIs. Every other module is labelled a diagnostic probe rather than quietly presented as a powered result

HONEST LIMITS

- The agentic anchor was withdrawn after publication. A harness subtlety (Inspect reads the model-visible tool description from the execute function's docstring) meant the poisoning payload never reached the model, so an n=20 table that had already been written up measured something else entirely.
- Both scorers miss the same paraphrase in the same direction. A completion that plainly describes the injection attempt scores zero transparency and inflates attack success, because the phrase list does not contain that wording.

# AI Health Journal

Local RAG Journal with a Measured Safety Floor

*A private journaling AI where the two claims that matter, recall and crisis safety, are measured, not promised.*

**0.968 Recall@3**  
RETRIEVAL

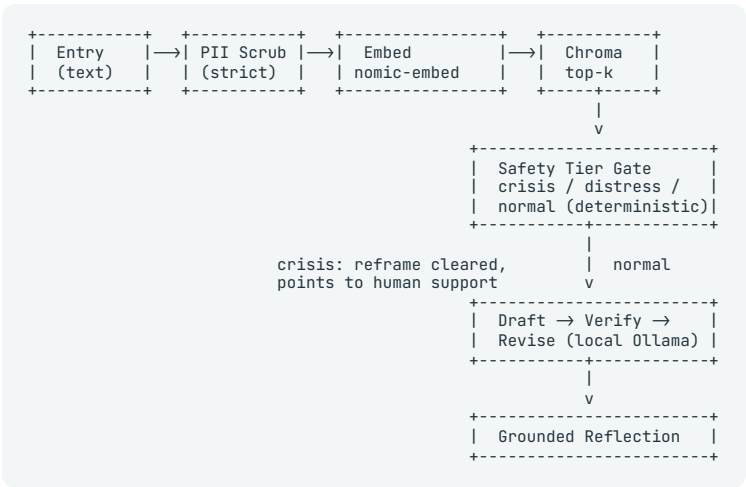
**1.000 sensitivity**  
CRISIS FLOOR

CASE STUDY [chunduri-aditya.github.io/Portfolio/work/ai-health-journal](https://chunduri-aditya.github.io/Portfolio/work/ai-health-journal)  
CODE [github.com/Chunduri-Aditya/journal-agent](https://github.com/Chunduri-Aditya/journal-agent)

### PROBLEM

A journaling assistant that reads back your own history has to do two things well, and both are easy to fake. It has to retrieve the entry that actually matters rather than the one that shares the most words, and it has to recognise when an entry describes a crisis rather than a bad week. Getting the second one wrong in either direction causes harm: miss it and someone in danger gets a cheerful reframe, over-trigger it and ordinary difficulty gets pathologised.

### ARCHITECTURE



### DECISIONS

#### Deterministic safety floor underneath the model, not inside it.

The tier gate cannot depend on an LLM call succeeding. Whatever else fails, the crisis path still has to hold. The rule is that whether a property is mechanically checkable decides whether it belongs in code or in the model. Cost: The floor is English-only and brittle to phrasing. Six known gaps are recorded and excluded from the headline rather than chased with an ever-growing regex, and two patterns were removed after live testing because they collided with ordinary usage.

#### Set each filter's breadth by the cost of its own false positive.

Missing a crisis is unrecoverable, so the crisis floor is deliberately over-broad and keeps false positives it could remove. The harsh-output filter runs the opposite way: a false positive there silently deletes a useful suggestion, so over-broad patterns would quietly gut the analysis. Cost: There is no single sensitivity dial to reason about. Each filter has to be argued and measured separately, and some false positives are kept on purpose.

### WHAT I BUILT

AI Health Journal is a local first journaling assistant whose two load bearing claims, does retrieval surface the right past entry and does the safety floor catch a crisis, are measured and reproducible offline rather than asserted.

### EVIDENCE

- Retrieval ablation across 4 strategies (dense MiniLM, BM25, hybrid RRF, dense nomic-embed-text) on a corpus where every query is tagged with the confusion it was built to induce
- The valence\_flip category went 0.667 to 1.000 on an embedder swap that also lifted full corpus Recall@3 to 0.968 and runs roughly twice as fast (26.6ms vs 58.0ms median per embedding)
- Two alternative fixes built, measured, and rejected: a score threshold (relevant and irrelevant distributions overlap completely) and a valence aware reranker (helped the weak embedder, actively hurt the strong one). Both kept documented rather than deleted, so they do not get re proposed

### HONEST LIMITS

- The external validation is unflattering and was kept. Scored against GoEmotions, the valence lexicon reached 0.551 overall accuracy where always guessing the majority class is the baseline, with negative recall at 0.260. A coverage fix moved it to 0.575 on an untouched split, and the log's own verdict is that this is the correct headline number and it is not a good one.
- Crisis sensitivity is optimistic by construction. The case set was authored with the detection patterns visible, so specificity is the more trustworthy half and an unbiased estimate needs a held-out set. The repo says so next to the number.

# Sourcewarden

Security-Gated n8n Workflow Orchestration

*A chatbot that helps you build n8n workflows, and only exposes what's actually safe to expose.*

2,256 rows

RETRIEVAL INDEX

Verified live

SECURITY HASHES

71 passing

TESTS

CASE STUDY

[chunduri-aditya.github.io/Portfolio/work/sourcewarden](https://chunduri-aditya.github.io/Portfolio/work/sourcewarden)

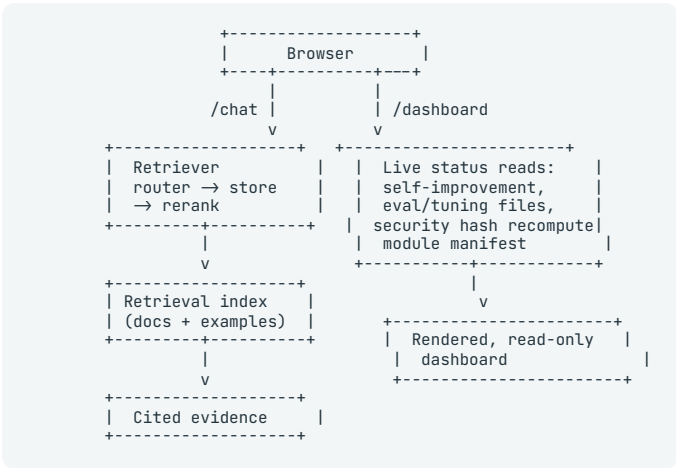
CODE

[github.com/Chunduri-Aditya/sourcewarden](https://github.com/Chunduri-Aditya/sourcewarden)

PROBLEM

An assistant that builds automation workflows has two ways to be dangerous. It can invent a node parameter that looks plausible and produces a workflow that fails in production, and it can take a live action against a real instance on the strength of its own say-so. This project treats both as gate problems rather than prompt problems: evidence has a fixed precedence order, and any live mutation is held behind a fail-closed, externally signed approval.

ARCHITECTURE



DECISIONS

**Reimplemented the hash check in Python instead of shelling out.** The existing hash generator is Node-only, write-capable, and maintainer-only. Giving a read-only backend any path to a write-capable script defeats the point of read-only, and reimplementing removes both the external binary dependency and the shell-injection surface. Cost: Two implementations of the same hashing rule now have to agree, and the repo does not record what that duplication costs to maintain.

**Self-improvement may propose, never apply.** The tuner and the controls that authorise it share one local trust boundary, so a local apply command would let a compromised tuner change both the behaviour and the evidence for it. The loop terminates at an external integrator instead. Cost: The loop cannot close. Every proposal sits permanently on hold, because the external signing infrastructure that would complete it does not exist in the repository.

WHAT I BUILT

Sourcewarden merges a retrieval index and a multi agent orchestration system for building n8n workflows behind a read only FastAPI layer that grounds every chat answer in cited evidence and verifies its own security controls live instead of shelling out to run anything itself.

EVIDENCE

- 2,256-row retrieval index across official n8n docs and community workflow examples, with an explicit evidence precedence: live instance schema, then official docs, then community examples, then untrusted references
- Six-role agent orchestration (supervisor, security firewall, skeleton architect, module builder, deviation monitor, eval tuner) gated by an Ed25519-signed run ledger, one-time nonces, and a 15-minute authorization window
- Deterministic secret-detection policy blocks any self-improvement feedback containing a PEM key, named credential, or GitHub/OpenAI token pattern, fail closed

HONEST LIMITS

- The flagship evaluation run executed zero behavioural cases. Its own result file records the reason: no approved behavioural runner or externally signed evaluator evidence was available, and structural validation is not a behavioural score. The audit reaches the same verdict independently and holds the proposal as not promotable.
- Live mutation against a real instance is never exercised end to end. It needs a connected instance plus an externally produced signed ledger, so the gate correctly denies by default and there is no in-repo demo of a successful signed mutation.

# MetaLearnML

Meta-Learned AutoML Ranking, Measured Against Its Baselines

*I built an AutoML ranker, then the benchmark that proved it does not save time, and I kept that result.*

**0.57 vs 0.02 p**

RANKING (META VS RANDOM)

**0.0% (CI 0 to 50)**

MEASURED SPEEDUP

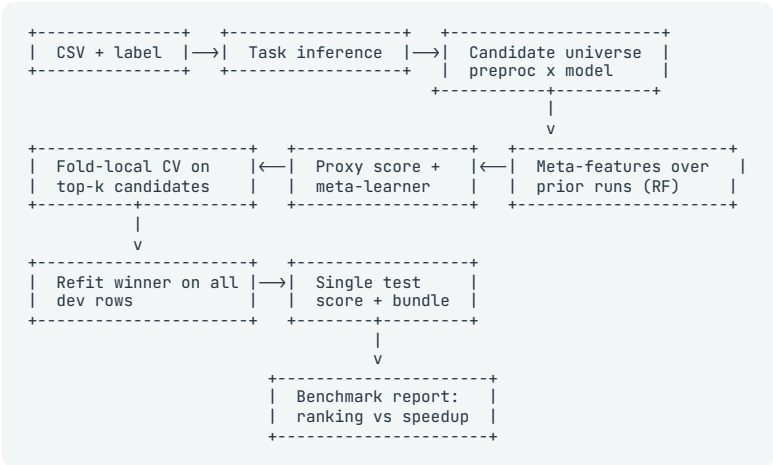
CASE STUDY [chunduri-aditya.github.io/Portfolio/work/metalearnml](https://chunduri-aditya.github.io/Portfolio/work/metalearnml)

CODE [github.com/Chunduri-Aditya/MetaLearnML](https://github.com/Chunduri-Aditya/MetaLearnML)

## PROBLEM

An AutoML system can rank candidate pipelines with a meta-learner trained on past runs, and the obvious claim is that better ranking finds a good pipeline in fewer evaluations. That claim is rarely tested against the baselines it needs to beat. This project builds the ranker and then builds the benchmark that decides whether the ranking actually saves any evaluations.

## ARCHITECTURE



## WHAT I BUILT

MetaLearnML is a tabular AutoML engine that ranks preprocessing by model candidates with a meta learner trained on past runs, next to a benchmark over 15 datasets built to measure whether that learned ranking actually beats proxy and random baselines. On end to end evaluation savings it does not, and the repo records that.

## EVIDENCE

- Candidate universe: up to 12 preprocessing strategies × 7 (classification) or 9 (regression) models, ranked by fast proxy evaluation plus an optional RandomForest meta-learner over prior-run meta-features
- Leakage controls: outer dev/test split before any encoder is fit, fold-local preprocessing, a single scored touch of the test partition, deterministic splits, SHA-256 content-addressed candidate identity, and a versioned meta-feature schema that refuses to load on drift
- Benchmark: 15 OpenML datasets (8 classification, 7 regression), outer seeds [13, 42, 97], 5 inner folds, 20 seeded candidate orderings, methods {exhaustive, proxy, meta, proxy+meta, random}

## DECISIONS

**Pre-register the success bar in code, then publish the failure.** The criterion is a boolean in the analysis script, evaluated automatically: a wholly positive bootstrap interval, at least 95% quality retention at budget 10, and regret no worse than 0.05. Deciding it after seeing results is how a null quietly becomes a win. Cost: It cost the project its headline claim. The generator wrote 'resume impact criterion not met' and the machine-readable summary records eligibility as false.

**Separate ranking quality from end-to-end savings.** The meta-learner genuinely ranks better than random, by median Spearman 0.5676 against 0.0216. That is not the same as reaching a good model in fewer evaluations, and conflating the two is the trap. Cost: Reporting both means publishing a real improvement next to a null result and refusing to let the first one stand in for the second.

## HONEST LIMITS

- The headline result is null and stays published. Median evaluation reduction against random is 0.0%, with a dataset-bootstrap 95% CI spanning 0% to 50% over 10,000 resamples, which does not establish a reduction. The repo states it does not claim a search speedup.
- Ranking quality and evaluation savings came apart. Median Spearman is 0.5676 for meta-only against 0.0216 for random, so the learned ranking genuinely orders candidates better; it just does not convert that into fewer evaluations. The gap between those two facts is the actual finding.

# twin

A Personal Agent Across Claude, Codex and Gemini, Behind Approval Cards

*One chat on my Mac, three models behind it, and nothing leaves on my behalf until I click.*

1,477

OFFLINE TESTS

646

MUTATION ROWS

CASE STUDY [chunduri-aditya.github.io/Portfolio/work/twin](https://chunduri-aditya.github.io/Portfolio/work/twin)

CODE Private repo, [access on request](#)

## PROBLEM

I wanted one agent that answers in my voice and builds things by handing each step to the model that suits it: Claude, GPT through Codex, and Gemini. The design problem is trust across those handoffs. No model should grade its own work, text a model read (a hostile email included) must not break out of the quote it arrives in, and nothing should reach another person on my behalf until I have seen exactly what will be sent.

## ARCHITECTURE

```

you: chat page (127.0.0.1) or voice (whisper.cpp on the Mac)
    |
    v
chat.py  converse hop: reply now, or start a job (typed flag or /do)
        opt in: lead.py decides one step at a time
                |
                v
                the job's task is your words, never a model's
        |
        v
loop.py  gemini digest (long input) → claude plan → claude draft
        → codex critique → claude revise (up to 2) → claude final
        routing.toml: model per role, $1 per run, $20 per day
                |
                v
                every model reply
        |
        v
taint.py typed shape check: accept as quoted data, or quarantine
        |
        v
        a write it wants to make
        |
        v
approval.py card bound to the sha256 of what you saw, 600 s,
        one click, or 8 characters of the digest typed back
        → gate.py  PreToolUse hook: approved arguments only, runs once

journal.py append only log of every step, rendered on the page
memory.py one markdown file per chat, sqlite FTS5 + local vectors
mcp.py    other agents start runs over stdio
  
```

## WHAT I BUILT

twin is a personal agent you chat with, typed or out loud. It answers right away or starts real work in the background, where Claude plans and drafts, Codex reviews every step with a typed verdict, and Gemini condenses long input. Every model reply is untrusted until a taint gate checks its shape, and every change to mail, calendar or documents waits on an approval card bound to a hash of exactly what was shown.

## EVIDENCE

- A /do job runs plan, draft, review, revise, final: Claude (Opus 5.5) plans up to 4 steps and drafts each, Codex reviews every draft with a typed verdict, and Claude revises up to 2 rounds. A live email job planned 1 step and took 4 calls, 15.9 s and \$0.064, with 1 of 1 step agreed (2026-09-28)
- 43 connector write tools, every one behind an approval card bound to a SHA256 of exactly what was shown. The first decision wins, a card expires after 600 s, and a PreToolUse hook rechecks it right before it runs once. Live: a one click Gmail draft in 7.7 s and its typed delete in 8.4 s (2026-09-28)
- 1,477 offline tests, all passing, and 646 mutation rows that break a guard on purpose; a guard whose test still passes counts as missing. The last full run of 620 rows left 1 alive, and that guard now has its test

## DECISIONS

**Drive the CLIs headless, not the APIs.** Claude and Codex run on subscriptions that already exist, Gemini on an AI Studio key, and all three CLIs return structured output, so each adapter calls its CLI headless, never drives a TUI and passes no bypass flag by default. Cost: On a Max login the dollars a CLI reports are its own estimates, not a bill, so the \$20 day cap paces use and stops a runaway rather than tracking real spend.

**Keep the working models text only.** The models that plan, draft and review get no tools and no MCP servers. Account reads and web search run in a separate restricted call with only the tools they need; denying every other server whole also cut the tool list from 117 to 47 and the probe's cost from \$0.47 to \$0.25. Cost: A lookup answer steered by a hostile mail still stays in the quoted chat history and could steer a later turn into proposing a change. The approval card, not the split, is what bounds that.

## HONEST LIMITS

- The taint gate checks shape, not meaning. Quoted text can still persuade a model that reads it, and one code guard is a grep over tokenized source that aliasing gets past.
- Meetings and phone calls are built but have never run live, and the browser voice page has not been tested with a real mic.

# AI RemixMate

Full-Stack DJ Engine with Research-Grade MIR

A DJ engine that mixes two songs into one clean transition, with 904 tests proving it.

904  
TESTS

-14 LUFS  
MASTERING

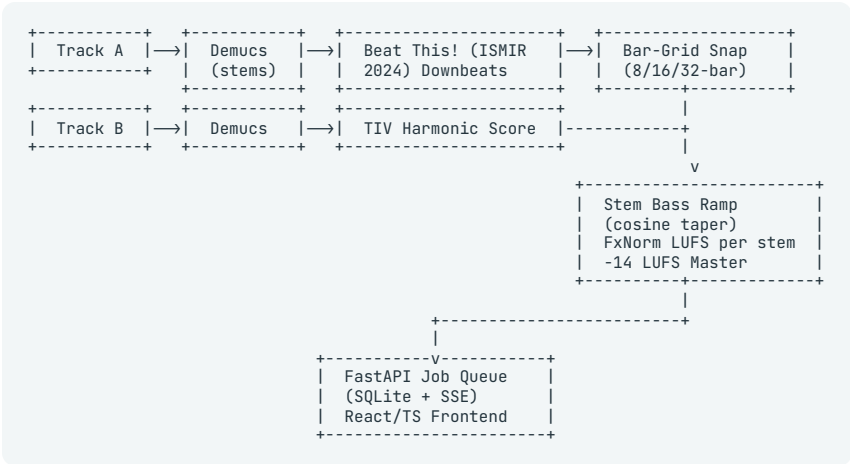
CASE STUDY [chunduri-aditya.github.io/Portfolio/work/ai-remixmate](https://chunduri-aditya.github.io/Portfolio/work/ai-remixmate)

CODE [Private repo, access on request](#)

WHAT I BUILT

AI RemixMate is a full stack DJ engine: a React/TypeScript frontend with SSE live streaming, a FastAPI async job queue, and a research grade MIR core (TIV harmonic scoring, Beat This! downbeat detection, CLAP 512-D semantic search), with 904 tests and mastering to -14 LUFS.

ARCHITECTURE



EVIDENCE

- FastAPI async job queue (SQLite write-through persistence) + React/TypeScript frontend, 10 pages, SSE live job streaming, Zustand state management
- TIV harmonic scoring (Bernardes et al. 2016 Tonal Interval Space) replacing psychoacoustic consonance approximation
- Beat This! (ISMIR 2024) downbeat detection with 8/16/32-bar grid snapping, replacing librosa beat\_track
- Cosine-taper stem bass ramp (no IIR bleed) + FxNorm per-stem-type LUFS normalization using corpus-derived targets
- CLAP 512-D semantic search (crate\_digger.py) + Essentia arousal/valence energy arc modeling for setlist planning
- rekordbox XML + Serato GEOB cue export to bridge output into professional DJ software

DECISIONS

**Replace librosa beat\_track with Beat This! (ISMIR 2024).** librosa gives beat times but misses downbeats. Phrase-boundary cue snapping to 8/16/32-bar grid requires knowing which beat is beat 1. Beat This! predicts this directly.

**TIV harmonic scoring alongside Camelot Wheel.** Camelot is discrete (adjacent or not). TIV gives a continuous compatibility score [0,1] from the Tonal Interval Space, so transition planning can rank candidates, not just filter them.

**Cosine-taper stem bass ramp instead of IIR shelving.** IIR filters have group delay and bleed on sustained bass notes. A cosine taper with derivative=0 at endpoints makes no audible click and hard-zeros Song A bass after the swap.

**CLAP 512-D semantic search over 35-D handcrafted embeddings.** 35-D features (BPM, key, energy) can't capture timbral similarity. CLAP embeds audio into a space shared with text, so 'find me something that sounds like X' actually works.

TRADEOFFS

- Beat This! (accurate downbeats, ~50 MB model) vs librosa (fast, weaker bar detection) — Beat This! wins for cue snapping accuracy
- TIV harmonic score (MIR literature, continuous) vs Camelot Wheel (DJ convention, discrete) — both exposed; TIV used for transition planning
- CLAP 512-D (semantic audio search, 300 MB download) vs 35-D music\_index (instant, keyword-only) — CLAP for search, 35-D as fallback
- True stem bass ramp vs IIR shelving filter — ramp eliminates IIR bass bleed on sustained notes

# taintgate

Turn-Scoped Containment for Coding Agents

Once a coding agent reads something hostile, it stops being allowed to act for the rest of that turn.

124

TESTS

10

MUTATIONS VERIFIED RED

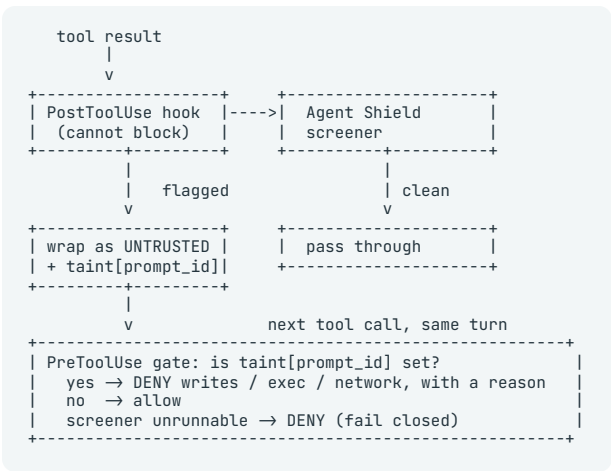
CASE STUDY [chunduri-aditya.github.io/Portfolio/work/taintgate](https://chunduri-aditya.github.io/Portfolio/work/taintgate)

CODE [Private repo, access on request](#)

## WHAT I BUILT

A Claude Code hook layer that screens tool results through Agent Shield, wraps flagged content as untrusted data before the model reads it, and denies every write, execute and network tool for the remainder of that turn, keyed by prompt\_id.

## ARCHITECTURE



## EVIDENCE

- Turn-scoped taint keyed by prompt\_id, surviving across separate hook subprocesses, so ingest in one tool call constrains every later call in the same turn
- Gate covers 14 tool names plus every mcp\_\* tool by prefix. The matcher was checked against the harness rather than assumed: a probe confirmed Bash and an MCP tool fire while Read does not, which a wildcard matcher would have hidden
- Fails closed. With the screener pointed at a path that does not exist, a WebFetch was denied with an explicit reason rather than quietly allowed
- Live deny rate of 1 denied in 10 gated calls, recorded with the allow path too, so the rate has a real denominator
- 124 tests, and ten mutations reintroduced as defects were each observed red then restored byte-identical
- Documents its own recall gaps rather than hiding them: as screened on 2026-09-17, secret-key assignments and a base64-piped-to-shell command scored allow low and are not wrapped. Those are gaps in the screener's ruleset, named in the README

## DECISIONS

**Key the taint on prompt\_id, not session\_id.** It is the only identifier that scopes to a single turn and survives across separate hook subprocesses. A probe confirmed a subagent's PreToolUse carries the parent turn's prompt\_id, so taint propagates into subagents rather than leaking around them.

**Fail closed when the screener cannot run.** A security layer that silently degrades to allow is worse than none, because it is trusted. An executable check denies writes and names the exact missing binary in the message.

**Publish the recall gaps in the README.** The layer wraps what the screener flags, which is not the same as everything dangerous. Stating the payloads that currently score allow low is the difference between a security tool and a security claim.

## TRADEOFFS

- PostToolUse cannot block, only rewrite, so quarantine wraps unconditionally when the guard is missing rather than pretending it screened something
- Turn scope rather than session scope: a session-wide block would make one bad web page end the session, while a turn-scoped one degrades to read-only and recovers
- Guards a cooperative model's mistakes, not an adversarial one. The run log is writable through Bash by means no gate here inspects, and the README says so rather than implying containment it does not provide

# Company Agents

An Executable Map of a Company's Roles

*Company functions down to role agents, with a verifier that refuses to call a blank job done.*

42 / 42

VERIFY STEPS

0

RUNTIME DEPS

CASE STUDY

chunduri-aditya.github.io/Portfolio/work/company-agents

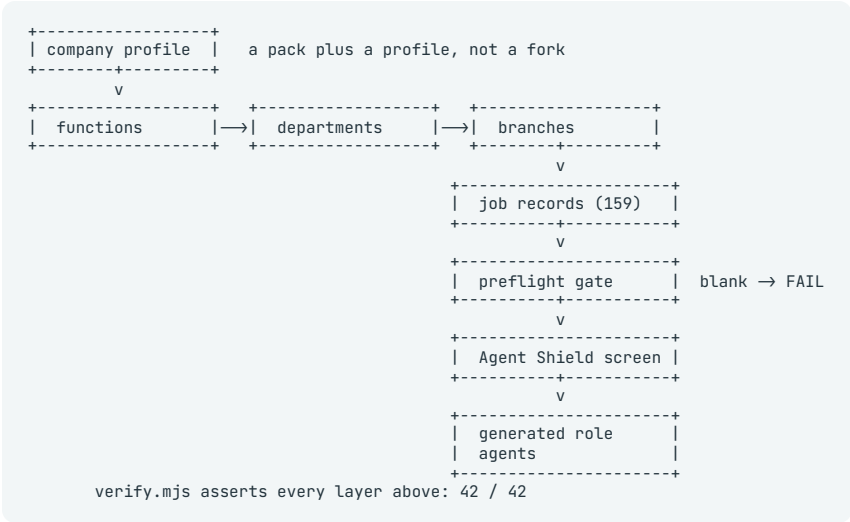
CODE

Private repo, [access on request](#)

WHAT I BUILT

An executable map of company functions to departments to branches to jobs to role agents, with a five-layer enforcement chain: job records, branch designs, a preflight gate, context screening through Agent Shield, and generated role agents.

ARCHITECTURE



EVIDENCE

- node scripts/verify.mjs passes 42 of 42 steps, exit 0, with the repository audit green
- Caught its own false success: a caption pass ran against blank inputs because 51 of 159 job records used a heading the runner did not read, and the run still exited 0. Fixed, and the verifier now fails on it
- Three mutations reintroduced as defects, each observed red then restored
- Zero runtime dependencies: Node builtins and relative imports only, tested with node --test
- Live model runs recorded with their failures, not just their successes: of three runs on a local 8B model, one completed with all four required outputs and two were blocked for inventing a URL
- Agent Shield wired in as a live dependency for context screening rather than named as a design intention

DECISIONS

**A company is a pack plus a profile, not a fork.** Forking the repo per company means every fix has to be applied N times and drift is guaranteed. Keeping the structure in records means a new company is data.

**Fail the run on a blank job rather than let it exit 0.** The failure that motivated this produced plausible output from empty inputs and reported success. An agent pipeline that cannot tell empty input from a hard task will confidently produce nothing.

TRADEOFFS

- Zero dependencies keeps the whole chain auditable and installable anywhere, at the cost of writing the runner and test scaffolding by hand
- Records and generated agents rather than hand-written agents: adding a company is configuration, but the generator becomes the thing that has to be right
- A preflight gate that fails on blank input costs runs that would previously have reported success, which is the point

**Record what the repo is not.** STATE.md carries the correction that this is not an operational multi-agent company, along with its open bugs. A skeleton described as a product is the failure mode this project exists to avoid.

# Persona RAG

Persistent Agent Memory Behind a Fail-Closed Privacy Boundary

*A context window is always too small. This decides what earns a slot in it.*

**k=5 · 3 sources**

CONTEXT BUDGET

**0.6 containment**

LEAK GATE

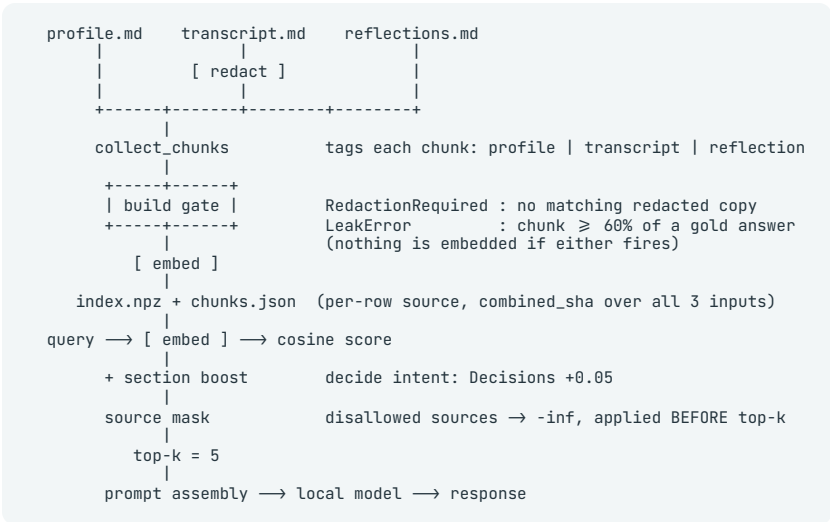
CASE STUDY [chunduri-aditya.github.io/Portfolio/work/persona-rag](https://chunduri-aditya.github.io/Portfolio/work/persona-rag)

CODE [Private repo, access on request](#)

## PROBLEM

A personalised agent accumulates far more material than a context window holds, so every turn is a selection problem: given a question, which few pieces of what the system knows about a person actually deserve to be in front of the model? Ranking by similarity alone answers that badly, because the most textually similar paragraph is often not the most useful one, and because some material should never reach the model regardless of how well it scores.

## ARCHITECTURE



## DECISIONS

**Mask sources before top-k, not after.** Filtering after ranking would let a disallowed chunk consume one of the five slots and then be dropped, silently shrinking the context. Setting masked rows to negative infinity before selection means the budget is always spent on admissible material. Cost: Scores are no longer comparable across filter settings, so a chunk's rank only means something relative to the sources currently allowed.

**Refuse to build the index rather than warn.** A redaction gate that only warns is a gate that gets skipped under time pressure, and the failure it prevents is unrecoverable: once raw transcript text is embedded, it is in the index. The build raises instead. Cost: One unredacted chunk blocks the entire build, which is deliberately obstructive during iteration.

## WHAT I BUILT

A persistent personalised agent whose retrieval layer tags every chunk with its origin (profile, interview transcript, or expert reflection), ranks candidates by cosine similarity with a per-section offset, and masks disallowed sources to negative infinity before top-k so a filtered-out chunk cannot re-enter on score alone. The index refuses to build at all while a real transcript lacks a matching redacted copy, or while any transcript chunk contains more than 60% of a gold evaluation answer.

## EVIDENCE

- Three source-tagged indexes over the profile, the redacted interview transcript, and expert reflections; every chunk and every index row carries its source
- Retrieval masks disallowed sources to negative infinity before top-k, so a source filter cannot be defeated by a high similarity score
- Section-aware ranking: a decide-intent query adds a +0.05 offset to chunks from the Decisions section, enough to change the cut only where scores are close

**Treat evaluation leakage as a build-time error.** If indexed material contains the gold answers, retrieval scores measure memorisation rather than retrieval, and the evaluation quietly stops meaning anything. A containment check over gold answers runs before anything is embedded. Cost: The 60% word-containment threshold is a heuristic. It will not catch a paraphrased answer, and it can fire on a passage that merely shares vocabulary.

# Smaller systems and coursework

Shorter builds without a full case study. Each links to its code.

## AkashicTree Automated Content Generation Pipeline

GENAI / PIPELINE · SHIPPED

AkashicTree is an agentic, model agnostic GenAI media pipeline that turns a single brief into text, image, and audio, coordinating local Ollama inference, Diffusers / FLUX.1, and ElevenLabs voice through one modular workflow.

- Single-brief → text + image + audio via one orchestration layer
- Local Ollama inference for text generation

Text · Image · Audio Modalities    Agentic, modular Workflow    Code: [Private repo, access on request](#)

## Model Behavior Lab Local LLM Evaluation Platform

EVAL / TOOLING · SHIPPED

Model Behavior Lab is a local Ollama based evaluation platform that benchmarks reasoning, hallucination, emotion alignment, and code correctness with repeatable runs and dashboards. It is the methodology that became the base for Agent Shield.

- Local Ollama-based evaluation — no external API dependency
- Benchmarks reasoning, hallucination, emotion alignment, and code correctness

4 eval axes Categories    Local Ollama Runtime    Code: [github.com/Chunduri-Aditya/Model-Behavior-Lab](https://github.com/Chunduri-Aditya/Model-Behavior-Lab)

## Attention Drift Detector On Device Attention Monitoring (No Video Leaves the Machine) CV / LOCAL-FIRST · SHIPPED

Attention Drift Detector classifies focus, drift, and absence in real time from head pose and iris gaze, nudges you after five continuous seconds of drift, and writes a session report. No video is ever recorded: only derived angles and labels reach disk.

- MediaPipe Face Mesh (478 landmarks) to head pose via solvePnP to iris based gaze estimation, then a rule based classifier with temporal smoothing over a 1 second sliding window
- Three states (focused, drifting, absent) with an OS notification firing after 5 continuous seconds of drift

478 landmarks Pipeline    52 · under 1s Tests    Code: [github.com/Chunduri-Aditya/attention-drift-detector](https://github.com/Chunduri-Aditya/attention-drift-detector)

## profile-rag Retrieval-Only Q&A Over This Site

RETRIEVAL / API · SHIPPED

profile-rag answers questions about this portfolio from its own content.ts, exported as 318 fact-sized chunks. BM25 and a dense Chroma index are fused by reciprocal rank, a cross-encoder reranks six candidates, and the best sentences of the top chunk come back with links to their source. A hand-written FAQ layer answers the canonical questions verbatim. Everything runs on CPU with ONNX models on a free host, so there is no API key and no token limit.

- Frozen 48-question eval bank with expected chunk ids: hybrid plus rerank puts the right chunk in the top three for 47 of 48 questions, against 40 of 48 for BM25 alone (2026-09-22)
- The eval floor was mutation-checked: removing the reranker's sort dropped the top-three hits to 46 of 48, and the floor was set above that value so the gate can actually fail

47 / 48 questions Recall@3    0 LLM calls    Code: [github.com/Chunduri-Aditya/profile-rag](https://github.com/Chunduri-Aditya/profile-rag)

## ChatDB Rule-Based Natural-Language-to-SQL CLI

DEV-TOOL / SQL · COURSEWORK

ChatDB loads a CSV into SQLite and turns a fixed grammar of plain English aggregate questions into SQL with regex pattern matching, not a model.

- CSV ingestion with pandas: column-name normalization, then dtype-based classification into datetime / measure / attribute columns to drive query planning
- Five named regex intent patterns (sum / average / min / max / count, each grouped-by) mapped to SQL GROUP BY templates

5 regex patterns Translation    pandas · SQLAlchemy Stack    Code: [github.com/Chunduri-Aditya/ChatDB](https://github.com/Chunduri-Aditya/ChatDB)

Beyond Attack Success Rate: Measuring Operator-Facing Transparency in LLM Agent Security

Sole-author preprint introducing a transparency-aware evaluation protocol on Inspect AI. Adds Transparency Rate (TR) as a third axis to the AgentDojo outcome matrix, with an anchored prompt-injection result (n=20, Wilson 95% CIs) across four models and diagnostic probes over six adversarial surfaces.

Read The Paper: [chunduri-aditya.github.io/Portfolio/Docs/Agent\\_Shield\\_Paper.pdf](https://chunduri-aditya.github.io/Portfolio/Docs/Agent_Shield_Paper.pdf) View on Zenodo: [doi.org/10.5281/zenodo.20789431](https://doi.org/10.5281/zenodo.20789431)  
GitHub: [github.com/Chunduri-Aditya/agent-shield](https://github.com/Chunduri-Aditya/agent-shield)

Wind Power Analysis Using Machine Learning in Wind Turbines

Hybrid forecasting combining TCN and KNN regression inside a Digital Twin architecture on Azure. IJRASET Vol 11 Issue VIII, Aug 2023. Second of four authors.

Read The Paper: [chunduri-aditya.github.io/Portfolio/Docs/Publication\\_Paper\\_Wind.pdf](https://chunduri-aditya.github.io/Portfolio/Docs/Publication_Paper_Wind.pdf)  
View Certificate: [chunduri-aditya.github.io/Portfolio/Docs/IJRASET\\_Certificate\\_Wind.pdf](https://chunduri-aditya.github.io/Portfolio/Docs/IJRASET_Certificate_Wind.pdf) DOI: [doi.org/10.22214/ijraset.2023.52452](https://doi.org/10.22214/ijraset.2023.52452)

Where my evals lied

Five eval scores that read as measurements and were not, with the test suite green each time. Each entry names the number, why it was wrong, and the commit that corrected it, so every claim can be checked at source.

Read The Post: [github.com/Chunduri-Aditya/agent-shield/blob/main/docs/posts/where\\_my\\_evals\\_lied.md](https://github.com/Chunduri-Aditya/agent-shield/blob/main/docs/posts/where_my_evals_lied.md)

SKILLS

Evaluation & Adversarial Testing

Inspect AI, AgentDojo, HarmBench, Red Teaming, Prompt Injection, Benchmark Design, Mutation Testing, Wilson CIs

LLM & Agent Systems

LangChain, LangGraph, Ollama, Hugging Face, RAG, Tool Use, Agent Orchestration, Context Engineering, MCP

ML & Modeling

Python, PyTorch, TensorFlow, Keras, Scikit-learn, NumPy, Pandas

Computer Vision & Audio

OpenCV, MediaPipe, YOLO, Segmentation, Homography, librosa, Demucs, CLAP, Diffusers

Backend & ML Systems

FastAPI, Flask, React/TypeScript, Docker, GitHub Actions, CI/CD, pytest, uv

Data & Retrieval

ChromaDB, pgvector, Vector Search, PostgreSQL, SQLAlchemy, SQLite, Plotly

GET IN TOUCH

I am looking for ML and AI engineering work: agent systems, evaluation infrastructure, and the parts of a product that still have to hold up after the demo.

Tell me what the system has to do and what breaks when it does not. Every number on this page traces to a command and a commit, and I will walk through any of them.

[chunduriaditya2@gmail.com](mailto:chunduriaditya2@gmail.com) · [chunduri-aditya.github.io/Portfolio](https://chunduri-aditya.github.io/Portfolio) · [github.com/Chunduri-Aditya](https://github.com/Chunduri-Aditya) · [linkedin.com/in/chunduriaditya](https://linkedin.com/in/chunduriaditya)